# EE1080/AI1110/EE2102 Probability and Random Processes

3 April, 2025

Please answer all the questions. The total marking is for 100. Rules for each question and split across the questions are described below. Please read the instructions carefully and do not wait until the deadline to work on the questions.

## General Instructions for Submission

The format, naming convention for the files to be submitted is described below. *Submissions that do not follow the format below will not be considered for evaluation.*

- Create a separate Python file for each problem. For instance, the file for problem one should be named `1_ee25btech00000.py`, where **ee25btech00000** should be replaced with your roll number. Follow a similar naming pattern for the remaining files. For some problems that have two parts the names would look like `1a_ee25btech00000.py`, `1b_ee25btech00000.py`
- create a undertaking_Alice.pdf (replace Alice with your first name) with an undertaking that reads

  *" I encoded this program myself, did not copy or rewrite the code of others, and did not give or send it to anyone else.*

  *Signature"*
- Place all the Python files along with the undertaking in a `.zip` archive named `ee25btech00000.zip`, where **ee25btech00000** should be replaced with your roll number.

**Additional points to note :**

- Clearly indicate the **functions** or **code modules** that correspond to each part of the question for easier evaluation.
- Wherever possible, structure your code into **functions** or **modules**. For example, consider creating one function to **generate random samples** and another to **calculate the expected value** of a random variable.
- Add **comments** to your code to improve readability.
- If you use a formula directly, please **cite the source** (e.g., a Wikipedia page or textbook section) for reference.
- Use a **random seed** (e.g., `random.seed(42)`) in relevant problems to ensure your results are reproducible during evaluation.
- Follow proper **indentation** and adopt **good coding practices** to ensure your code is clean, organized, and easy to understand.

## 1 Gambler's Ruin

We are looking at a problem where a Gambler starts with $S$ rupees and at each game he can either win a rupee or lose a rupee with probabilities $p$, $1-p$ respectively. He continues to play until he accumulates a sum of UB or if he exhausts all his money. In this problem, you are expected to simulate the Gambler's ruin problem $N$ times. For each run of the problem, the number of games played by the Gambler can vary depending on when he accumulates a sum of $UB$ or exhausts all his money.

The amount of money available with the Gambler after the $t$-th game is denoted as $X_t$ and it is given by:

$$X_t = S + \sum_{i=1}^{t} Z_i$$

where $Z_i$ is a random variable such that:

$$Z_i = \begin{cases} 1 & \text{with probability } p \\ -1 & \text{with probability } 1-p \end{cases}$$

Capture the number of games that Gambler participates in across all the $N$ runs of this experiment. You are also required to find the expected value of the number of games played before the Gambler stops and probability of the Gambler winning. The next section describes how to do that.

### 1.1 Expected Number of Games

Let $E_i$ be the expected number of games played by the Gambler given he starts with capital $i$ for $i \in [1, U-1]$. Clearly we can assume $E_0 = E_U = 0$ as starting conditions and it can be seen that $E_i$ follows the following recursive relationship.

$$E_i = 1 + pE_{i+1} + (1-p)E_{i-1}$$

. Solve for $E_i$'s by solving the following linear equations:

$$\underbrace{\begin{bmatrix} 1 & -p & & & & \\ -(1-p) & 1 & -p & & & \\ & -(1-p) & 1 & -p & & \\ \vdots & & \ddots & \ddots & \ddots & \\ & & & & -(1-p) & 1 \end{bmatrix}}_{M} \begin{bmatrix} E_1 \\ E_2 \\ E_3 \\ \vdots \\ E_{U-1} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}.$$

### 1.2 Probability of Winning

Similarly, the probability of winning can be recursively expressed as:

$$P_i = pP_{i+1} + (1-p)P_{i-1} \text{ for } i \in [1 : U-1]$$

. where $P_0 = 0, P_U = 1$. $P_i$'s can be solved by solving following linear equations:

$$M \begin{bmatrix} P_1 & P_2 & P_3 & \cdots & P_{U-1} \end{bmatrix}^T = \begin{bmatrix} 0 & 0 & 0 & \cdots & p \end{bmatrix}^T$$

### 1.3 Requirements

- Input requirements: The four parameters $S, p, U, N$ are input in the order as described.
- Commands you should test with:
  1. `python 1_ee25btech00000.py 10 0.5 50 1000`
- Output Requirements
  1. Sample mean of the number of games before the Gambler stops and the expected number of games before the Gambler stops.
  2. Fraction of times the Gambler wins and the probability that a Gambler wins.
  3. Sample mean of the number of games given that Gambler wins and Sample mean of the number of games given that the Gambler looses.

### 1.4 Grading

This question accounts to 20 marks.

## 2 Sampling techniques

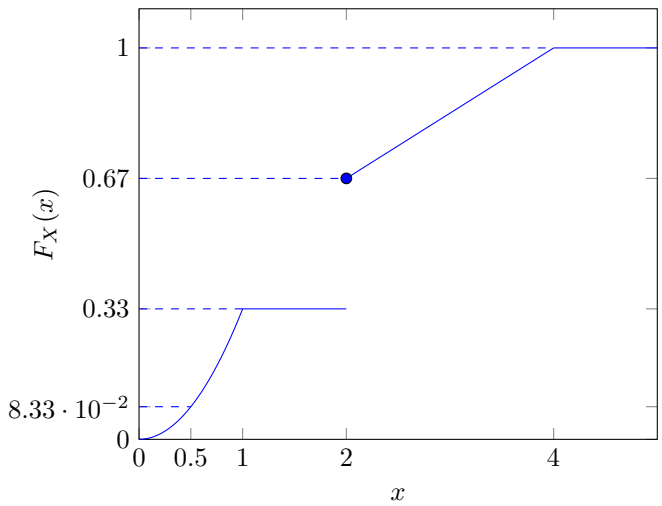
This has two parts, one that uses inverse sampling and the other that uses rejection sampling techniques.

## 2.1 Inverse Sampling

In this problem, you'll have to use $N$ uniform $[0, 1]$ random variable samples to:

1. generate $N$ samples of a Geometric($p$) random variable.
2. generate $N$ samples of an exponential($\lambda$) samples.
3. generate $N$ samples of a random variable $X$ with cumulative distribution function (CDF ), $F_X(x)$ given below:

$$F_X(x) = \begin{cases} x^2/3 & x \leq 1 \\ 1/3 & 1 < x < 2 \\ (x+2)/6 & 2 \leq x \leq 4 \end{cases}$$




### 2.1.1 Requirements

1. Input arguments to the this program:
   (a) first argument is mode it indicates, if it is 0, then your program should output Geometric samples, if 1, exponential samples and if 2, samples of $X$.
   (b) second argument is the samples file name. uniform_samples.txt test file is provided in the folder that you can use to test your code.
   (c) third argument is the parameter $p$ for mode 0 and parameter $\lambda$ for mode 1.
2. Example commands that you should test with:
   (a) python 2a_ee25btech00000.py 0 uniform_samples.csv 0.25
   (b) python 2a_ee25btech00000.py 1 uniform_samples.csv 0.5
   (c) python 2a_ee25btech00000.py 1 uniform_samples.csv 5.5
   (d) python 2a_ee25btech00000.py 2 uniform_samples.csv
3. Output of this program.
   (a) For all the modes, the input file needs to be read to get the uniform samples and the number of samples $N$ need to be determined.
   (b) For all the modes you are expected to save the sample outcomes to a an output file with following naming convention:
      i. Mode 0: invsampling_mode0_{p_value}_<name>.csv. For example command shown in 2a, you should have the output file name as invsampling_mode0_p25_ee25btech00000.csv
      ii. Mode 1: invsampling_mode1_{$\lambda$_value}_<name>.csv. For example command shown in 2c, you should have the output file name as invsampling_mode1_5p5_ee25btech00000.csv.

iii. Mode 2: invsampling_mode2_<name>.csv. For example command shown in 2d, you should have the output file name as invsampling_mode2_ee25btech00000.csv.

(c) The following extra outputs have to also be generated.
Text outputs will go to invsampling_out_<name>.txt, example invsampling_out_ee25btech00000.txt.

i. For Mode 0, after the Geometric samples are generated from the uniform samples read from the file, you need to compute sample mean and append to the output text file.

ii. For Mode 1, after the exponential samples are generated from the uniform samples, you need to plot histogram with bin size chosen to be $\sqrt{N}$ (see Stanley Chan, Chapter 3.2.5). Store in the format invsampling_mode1_hist_{p_value}_<name>.png (for example commans shown in 2c, file name should be invsampling_mode1_hist_5p5_ee25btech00000.png).

iii. For Mode 2, (1) print the number of times 2 appears in the the samples of $X$ generated from uniform samples (append to the output file in newline) and (2) plot a histogram of the samples of $X$ using bin size to be $\sqrt{N}$ and store in the format invsampling_mode2_hist_<name>.png (example invsampling_mode2_hist_ee25btech00000.png)

## 2.2 Gaussian sampling

Using inverse sampling techniques is difficult for gaussian sampling due to the lack of easily findable CDF. Therefore, we will use other methods to generate gaussian samples.

### 2.2.1 Rejection sampling

Rejection sampling uses techniques to generate random samples from a PDF $f(x)$ which is our target distribution using samples from another distribution $g(x)$ (which can easily be sampled) and accepting/rejecting them against some criteria.

Consider the following constraint on $f(x)$ and $g(x)$,

$$\frac{f(x)}{g(x)} \leq c \quad \forall\, x \in \mathbb{R}$$

We have the following steps for sampling a random variable $X$ having density $f(x)$,

1. Sample a random variable $Y$ with density $g$ and another random variable $U$ which is uniform from $(0, 1)$.
2. If $U \leq f(Y)/cg(Y)$ set X = Y, otherwise return to step 1 and sample again.

Please refer to the following reference for the proof and other various quirks of this method: Introduction to probability models (by Sheldon M. Ross) 10th edition, section 11.2.2 – Rejection sampling.

For sampling a standard normal random variable $Z \sim N(0, 1)$, we first sample a random variable which the absolute value of Z ($X = |Z|$) which has the following density function,

$$f_X(x) = \frac{2}{\sqrt{2\pi}} e^{-x^2/2} \ , \quad 0 < x < \infty$$

Here we may have an arbitrary selection of $g(x)$ for which $f(x)/g(x)$ is bounded by some constant for all $x$. Generally, $g(x)$ is taken as exponential with $\lambda = 1$ (note that here g(x) has nothing to do with $X$ itself, it is just a distribution function),

$$g(x) = \begin{cases} e^{-x}, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Finding $c$,

$$\frac{f(x)}{g(x)} = \sqrt{2e/\pi} \exp\{-(x-1)^2/2\} \leqslant \sqrt{2e/\pi} = c$$

Following are steps for producing a random standard normal sample,

1. Sample independent random variables $Y$ and $U$, $Y$ being exponential with rate 1 (distribution $g$) and $U$ being uniform on $(0, 1)$. (use scipy/numpy for these samples)
2. If $U \leq \exp\{-(Y-1)^2/2\}$, set $X = Y$, otherwise return to step 1.

However, we are not done here, as we still have to convert $X$ back to the standard normal $Z$ which can be easily done as $Z$ is equally likely to be $X$ or $-X$ and one of them can be chosen with equal probability.

Let the number of iterations after which you accept a sample be given by $N = \min\{n : U_n \leq f(Y_n)/(cg(Y_n))\}$ where $U_n, Y_n$ be the samples at $n^{\text{th}}$ iteration. You will observe that $N$ is a geometric random variable with mean $c$.

#### 2.2.2 Polar Transform

Polar transform is also a great way to generate gaussian samples, in particular you generate exponential samples (with rate 1/2) which correspond to the polar radius $R^2$ and uniform $(0, 2\pi)$ samples (use numpy/scipy for these samples) which correspond to polar angle $\theta$ and then transform it to the standard normal samples. Please refer to Introduction to probability models (by Sheldon M. Ross) 10th edition, section 11.3 – Special Techniques for Simulating Continuous Random Variables, in particular, 11.3.1 – The Normal distribution.

In both of the above methods, a general gaussian with some mean and variance can easily be generated using shifting and scaling of standard normal samples.

#### 2.2.3 Requirements

1. Input arguments to this program:
   (a) first argument is mode, mode 0 indicates that you should generate standard normal samples from rejection sampling method, mode 1 indicates that you should generate standard normal samples using polar transform.
   (b) second argument and third arguments are mean and variance of the samples (for both the modes).
   (c) last argument for each mode is the number of standard normal samples $M$ that have to be generated.
2. Example commands that you should test with:
   (a) python 2b_ee25btech00000.py 0 1 0.1 5000
   (b) python 2b_ee25btech00000.py 0 1 0.1 15000
   (c) python 2b_ee25btech00000.py 1 1 0.1 5000
   (d) python 2b_ee25btech00000.py 1 1 0.1 15000
3. Output of this program
   (a) Histogram of the samples (with bin size $\sqrt{M}$) is to be plotted which has to be overlayed on top of gaussian (with mean and variance given) PDF curve and is to be saved as gauss_sampling_hist_<mode>_<name>.png (for example gauss_sampling_hist_0_ee25btech00000.png). This is to be done for each mode.
   (b) Plot the histogram for the number of iterations $N$ (which is a random variable). As discussed in theory, you should ideally get an geometric with mean $c$. Overlay the histogram with the PDF curve of geometric with mean $c$. Histogram (with bin size $\sqrt{M}$) to be saved as gauss_iterpdf_hist_<mode>_<name>.png (for example gauss_iterpdf_hist_0_ee25btech00000.png). This is only to be done for mode 0.

### 2.3 Grading

This question accounts to $20 + 20 = 40$ marks with a split of (5+5+10) for part a and $15 + 5$ for part $b$.

## 3 Bertrand's Paradox

Bertrand's paradox problem is usually presented in the first lecture of a probability course to motivate why one needs to formally define probability model through sample space, event space and the probability rule trio $(\Omega, \mathcal{F}, P)$.

Suppose one is interested in finding the probability that a randomly chosen chord of a unit radius circle has length greater than that of the side of an equilateral triangle embedded in it. Depending of how the experiment of choosing a chord is done, the answer could be either 1/3 or 1/2 or 1/4. As part of this assignment you will be simulating all these three experiments.

Assume that the unit radius circle is centered at $(0, 0)$.

## 3.1 Mode 0: Angle made by chord w.r.t a tangent at one fixed end is equally likely

In this method, it is assumed that the end point of the chord is equally likely to be anywhere on the perimeter of the circle. So without loss of generality we will assume one end point is at $(0, 1)$. The experiment is done by picking angle the chord makes with tangent passing through $(0, 1)$ to be equally likely.

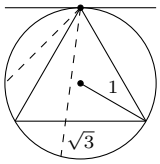



Figure 1: The dashed line is a chord that makes an angle $\Theta$ with tangent passing through the top of the circle. It can be seen that the chord has length $\geq \sqrt{3}$ iff $\Theta \in [60 : 120]$ from the figure above.

Let $X$ be the random variable representing the chord length. Determine it as a function of $\Theta$ from the geometry described above. Now generate $N$ uniform random samples of $\Theta$ in $[0 : \pi]$ and use them to generate samples for chord length. In this case, the probability that chord length is greater than $\sqrt{3}$ is $P(X \geq \sqrt{3}) = P(\Theta \in [\pi/3 : 2\pi/3]) = 1/3$.

### 3.1.1 Output requirements

1. Print the fraction of the chord length samples that are greater than $\sqrt{3}$.
2. Plot the histogram of the chord length samples. Use the square root rule to choose the bin size i.e., number of bins is $\sqrt{N}$.

## 3.2 Mode 1: Distance of the chord from center is equally likely

In this method, it is assumed that the distance from the center of the chord to the circle center is equally like to be in $[0, 1]$. Assuming the angle made by the chord with x-axis is equally likely to be $[0 : \pi]$, we consider that the chord is parallel to x-axis and appears below the x-axis without loss of generality.

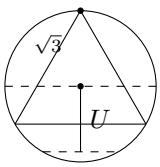



Figure 2: The dashed line is the chord is at distance $U$ from the center. $U \sim \text{Uniform}[0, 1]$.

Determine the chord length $Y$ as function of $U$ in this case. Now generate $N$ uniform random samples of $U$ in $[0 : 1]$ and use them to generate samples of $Y$ for chord length. In this case, the probability that chord length is greater than $\sqrt{3}$ is $P(Y \geq \sqrt{3}) = P(U \in [0 : 1/2]) = 1/2$.

### 3.2.1 Output requirements

1. Print the fraction of the chord length samples that are greater than $\sqrt{3}$.
2. Plot the histogram of the chord length samples. Use the square root rule to choose the bin size i.e., number of bins is $\sqrt{N}$.

## 3.3 Mode 2: Center of the chord is equally likely within circle

Here it is assumed that the center of the chord is equally likely to be a point in unit circle. From the description of the previous mode, we know that if distance of the chord center from the center of circle is $\leq 1/2$, then the length of chord is $\geq \sqrt{3}$. Here, the probability that the center of chord is at a distance $\leq 1/2$ from circle center is given by the area of smaller circle with radius $1/2$ i.e., the probability is $1/4$ in this case.

Given $(X, Y)$ is uniformly random within a circle of radius 1 centered at $(0, 0)$. The chord length is determined by $\sqrt{X^2 + Y^2}$ which is the distance of chord's center from the circle center. Find the CDF of $R = \sqrt{X^2 + Y^2}$. Use CDF function $F_R(r)$ together with the fact that $F_R^{-1}(U)$ is a random variable with CDF $F_R(r)$ given that $U$ is uniform random to generate samples of $R$. Now determine chord length $Z$ as a function of $R$, the distance of chord from the circle center. Apply this to get the chord length samples i.e., samples of $Z$.

#### 3.3.1 Output requirements

1. Print the fraction of the chord length samples that are greater than $\sqrt{3}$.
2. Plot the histogram of the chord length samples. Use the square root rule to choose the bin size i.e., number of bins is $\sqrt{N}$.

### 3.4 Requirements

1. Input requirements
   (a) first argument indicates mode. It can take values $0, 1, 2$.
   (b) second argument is the number of samples $N$
2. Example commands that you should test with:
   (a) python 3_ee25btech00000.py 0 1000
   (b) python 3_ee25btech00000.py 1 1000
   (c) python 3_ee25btech00000.py 2 1000

   Increase $N$ to validate that the probability of chord length exceeding $\sqrt{3}$ matches with 1/3, 1/2, 1/4 for modes 0, 1, 2 respectively.

### 3.5 Grading

This question counts to 35 marks with split of 10+10+15 across the three modes.

## 4 Beta Distribution

This question has two parts. First part looks at ordered statistics where Beta distribution appears naturally and the second one is a Bayesian update process where Bias of coin is updated as you observe coin tosses.

### 4.1 Ordered Statistics

Puchku organizes a party for 6pm, inviting 10 friends. She loves to play Dungeons and Dragons that requires at least 4 players (besides her for it to be fun). Everyone is excited, so as soon as the 4th friend has arrived, she is starting the game. Her friends arrive in a uniform distribution between 6 and 7pm (and no one arrives before 6, but everybody arrives until 7pm). Let $X$ denote the ratio of the starting time in the interval $[6, 7]$ (ie. $X$ is a random variable with value between 0 and 1 i.e., if the fourth friend arrived at 6.30pm then $X = 0.5$). Write a function simulating $X$. "The idea here is that to generate $N$ samples of $X$. This can be achieved by generating $10 * N$ uniform $[0, 1]$ samples and placing them in an $N \times 10$ matrix followed by sorting each row of the matrix in ascending order. The fourth column in the sorted matrix corresponds to $N$ samples of $X$."

Consider the following two density functions:

$$f_a(x) = \frac{10!}{3!6!} x^3 (1 - x)^6, \quad 0 \le x \le 1,$$
$$f_b(x) = \frac{10!}{4!5!} x^4 (1 - x)^5, \quad 0 \le x \le 1.$$

1. Input arguments to the this program:
   (a) first argument is number of samples $N$
2. Example commands that you should test with:
   (a) python 4a_ee25btech00000.py 10000

3. Output requirements

   (a) Plot the histogram of samples of $X$. Consider 100 bins of size 0.01 each. Also plot the two pdfs shown above in the same figure. Use normed=1 for histogram plot so that it can be compared with the pdfs.

   (b) Print $a$ if the histogram looks close to $f_a$ otherwise print $f_b$.

### 4.2 Bayesian update

Aditya finds a peculiar coin and wants to estimate how biased it is towards landing on heads. He performs $m$ separate batches of coin tosses, where each batch consists of $n$ tosses. The number of heads observed in each batch is recorded as a space-separated list of $m$ integers in a text file. Instead of analyzing all the data at once, he decides to update his belief about the coin's bias iteratively by reading the data one batch at a time.

Let $\Theta$ denote the distribution of the coin's bias. Aditya's initial prior belief is that the bias is uniformly distributed between 0 and 1. For a given batch of $n$ tosses, the number of observed heads, $Y$, follows a Binomial distribution:

$$Y \mid (\Theta = \theta) \sim \text{Binomial}(n, \theta).$$

Write a program to simulate this iterative update process. The goal is to use Bayes' theorem to analytically determine the posterior distribution of the bias after observing each batch of $n$ tosses, starting with the uniform prior. You will then use this newly calculated posterior distribution as the prior for the next batch, repeating this sequential update until all the data is processed.

The Uniform$(0, 1)$ distribution is mathematically equivalent to a Beta distribution with parameters $\alpha = 1$ and $\beta = 1$. Due to the property of conjugate priors, updating a Beta prior with a Binomial likelihood yields a posterior distribution that is also a Beta distribution, but with updated parameters. To plot the probability density function (PDF) at each step, you must mathematically derive how the parameters $\alpha$ and $\beta$ are updated after observing a batch of data. Once derived, you may use built-in Python functions (such as `scipy.stats.beta`) to compute and plot the PDFs. Pay close attention to how the variance of the distribution changes as the number of iterations increases.

1. Input arguments to this program:

   (a) First argument is the name of the data file containing $m$ space-separated integers, each representing the number of heads observed in that specific iteration. There will be a total of $m$ iterations.

   (b) Second argument is the batch size $n$.

2. Example commands that you should test with:

   (a) `python 4b_ee25btech00000.py coin_data.txt 20`

3. Output requirements:

   (a) Plot the probability density function (PDF) of the updated distribution of $\Theta$ for each iteration ($m$ iterations in total) on the same figure so that the progression can be observed.

   (b) On a separate figure, plot the variance of $\Theta$ against the iteration number (1 through $m$) to visualize how the variance decreases with more data.

   (c) Print the final mean and variance of the distribution after all data has been processed.

### 4.3 Grading

This question accounts to 30 marks with (10+20) split.

## 5 Queuing Theory: Wait Times

In this problem, you can think of "packets" simply as tasks or requests arriving to a system that can be handled only one at a time. For example, these could be customers lining up at a billing counter, food orders arriving in a restaurant kitchen, or data requests reaching a server on the internet. This is a single-server queuing system in which packets arrive over time and are served one at a time in a First-In-First-Out (FIFO) manner.

You are required to simulate this system and study how the average waiting time by packets in the system evolves as the number of packets increases. The inter-arrival times between any two packets are given by random variables $X_1, X_2, \cdots, X_n$. We will assume that $X_i$'s are i.i.d and exponentially distributed with rate $\lambda$.

The service times of each of the $n$ packets are assumed to be $Y_1, Y_2, \cdots, Y_n$ and $Y_i$'s are assumed to be i.i.d. We will consider two cases for the service time distribution (1) $Y_i \sim$ exponential($\lambda$) and (2) $Y_i \sim$ Uniform($[0, \frac{2}{\mu}]$).

As part of this problem, you need to generate $N$ samples for inter-arrival times and $N$ for service times. Using these times, you need to compute the waiting time for each packet. Note that service starts for a packet only after the packets that arrived earlier than that are serviced. The wait time needs to include both the service time and the wait time within a queue before service starts. This can also be seen as wait time, `T(k) = departure time(k) - arrival time(k)` for $k$-th packet. Clearly arrival time of $k$-th packet is $\sum_{i=1}^{k} X_i$

You will be required to compute a running average of the wait-times

$$\bar{T}(k) = \frac{1}{k}\sum_{i=1}^{k} T_i$$

for $k = 1, 2, \ldots, N$ and plot it with respect to $k$.

1. Input Requirements: The five inputs correspond to $N$, $\lambda_1, \lambda_2, \lambda_3$, $\mu$.
2. Check your implementation using the following commands
   (a) python 5_ee25btech00000.py 1000 0.1 2.9 10 3
   (b) Note that the $\lambda_i$'s here are such that $\lambda_1 < \mu$, $\lambda_2 \approx \mu$ and $\lambda_3 > \mu$
3. Output requirements:
   (a) Two figures. One figure that includes three plots of the running average system time $\bar{T}(k)$ versus $k$ for each of the three $\lambda$ values, assuming $Y_i$'s distribution is exponential.
   (b) Second figure that includes three plots of the running average system time $\bar{T}(k)$ versus $k$ for each of the three $\lambda$ values, assuming $Y_i$'s distribution is uniform.
   (c) Clearly label the axes and provide a legend indicating the values of $\lambda$'s and $\mu$.
4. Observations:

   Based on your simulations, answer the following in the comments of your code for the exponential and uniform cases.

   (a) Does the running average system time converge to a constant or keep increasing as $k$ grows?
   (b) How does the relationship between $\lambda$ and $\mu$ affect the stability of the queue?

### 5.1 Grading

This question accounts to 30 marks.

## 6 Central Limit Theorem

In this problem, you'll have to first:

1. generate $N \times n$ samples of a Bernoulli($p$) random variable or;
2. generate $N \times n$ samples of a Geometric ($p$) random variable or;
3. generate $N \times n$ samples of an exponential($\lambda$) samples.
4. generate $N \times n$ correlated samples, described through parameters $p, q$ as follows:
   (a) The first column comprising of samples $X_{i,1}$ for $i \in [N]$ are generated from Bernoulli($\frac{p}{p+q}$).
   (b) Samples of second column, $X_{i,2}$ are dependent on samples of first column and are sampled as follows.

$$\begin{aligned} X_{i,2} \mid X_{i,1} = 0 &\sim \text{Bernoulli}(p), \\ X_{i,2} \mid X_{i,1} = 1 &\sim \text{Bernoulli}(1-q). \end{aligned}$$

   (c) Similarly samples of $j$-th column are dependent on samples of $j-1$-th column and are sampled as follows:

$$\begin{aligned} X_{i,j} \mid X_{i,j-1} = 0 &\sim \text{Bernoulli}(p), \\ X_{i,j} \mid X_{i,j-1} = 1 &\sim \text{Bernoulli}(1-q). \end{aligned}$$

You can use available python libraries (*numpy.random*, *scipy.stats*) to generate these samples. Let $X_{i,j}$ be the sample at $i$-th row and $j$-th column. Collect the row averages to a vector $Y_i$ where:

$$Y_i = \frac{\sum_{j=1}^{n} X_{i,j}}{n}$$

- Plot the histogram corresponding to the $N$ averages along with the pdf of normal distribution with mean $\mu$ and $\sigma^2/n$ on the same figure. $\mu$ and $\sigma^2$ are the mean and variance of the random variable whose samples are being generated, respectively. *Scale the histogram so that the visually pdf and histogram can be compared. Try out normed=1 for histogram. The normal distribution needs to be plotted only for the first three modes.*

### 6.1 Input/Output Format

1. Input arguments to the this program:
   (a) first argument is the mode the program is in, if it is 0, then your program should generate Bernoulli samples, if 1, Geometric samples, if 2, exponential samples and if 3, correlated samples.
   (b) second and third arguments are $n$ and $N$ respectively
   (c) fourth argument is the parameter of the distribution i.e., if mode is 0 or 1 it is $p$, if mode is 2 it is $\lambda$ and if mode is 3 the fourth and fifth arguments are $p$, $q$ respectively.
2. Example commands that you should test with:
   (a) `python 6_ee25btech00000.py 0 100 10000 0.5`
   (b) `python 6_ee25btech00000.py 0 5 10000 0.5`
   (c) `python 6_ee25btech00000.py 1 100 10000 0.2`
   (d) `python 6_ee25btech00000.py 1 5 10000 0.6`
   (e) `python 6_ee25btech00000.py 2 100 10000 5`
   (f) `python 6_ee25btech00000.py 2 5 10000 5`
   (g) `python 6_ee25btech00000.py 3 100 1000 0.5 0.45`
   (h) *please check visually that with increase in $n$ value, the variance seen in histogram reduces and that the histogram resembles closer to that of normal pdf as $n$ increases.*
3. Output of this program.
   (a) figure with two plots, one that corresponds to histogram of sample means and the other with PDF of the Gaussian R.V with mean $\mu$ and variance $\sigma^2/n$ computed from the parameters of the distribution ($p, \lambda$ depending on the mode) and $n$. (Gaussian pdf plot is needed only for modes $0, 1, 2$)

### 6.2 Grading

This question accounts to 25 marks with 10 marks for the last mode and 5 each for the rest.

## 7 MMSE Estimation (Maximal Ratio Combining)

Let $W_1, W_2, \cdots, W_n$ be i.i.d normal random variables. $X$ is normal random variable with mean $\mu_0$ and variance $\sigma_0^2$ and $W_i$ is normal random variable with mean $\mu_i = 0$ and variance $\sigma_i^2$. Assume that $X, W_1, \cdots, W_n$ are independent. Let:

$$Y_i = h_i X + W_i$$

for all $i = 1, 2, \cdots, n$. You are given $Y_1, Y_2, \cdots, Y_n$ and are expected to estimate $X$ from them. Find the MMSE estimator $E[X|Y_1, Y_2, \cdots, Y_n]$ for this problem in terms of $Y_1, Y_2, \cdots, Y_n$ and the problem parameters $\mu_i, \sigma_i^2$ for $i = 0, 1, 2, \cdots, n$. Note here that:

$$\begin{aligned}
f_{Y_i|X}(y_i|x) &= f_{W_i}(y_i - x) \quad \text{(check why this is true)} \\
f_{Y_1,\cdots,Y_n|X}(y_1, \cdots, y_n|x) &= \prod_{j=1}^{n} f_{Y_j|X}(y_j|x) \text{(check why this is true)} \\
f_X(x) &= \frac{1}{\sqrt{2\pi\sigma_0^2}} e^{-\frac{1}{2\sigma_0^2}(x-\mu_0)^2}, \quad f_{W_i}(w) = \frac{1}{\sqrt{2\pi\sigma_i^2}} e^{-\frac{1}{2\sigma_i^2}(w)^2}
\end{aligned}$$

Use them to find $f_{X|Y_1,Y_2,\cdots,Y_n}(x|y_1,\cdots,y_n)$ and $E[X|Y_1,Y_2,\cdots,Y_n]$ (refer to Chapter 8 of Reference 3). ( You can also solve for linear MMSE estimate using $n$ measurements for this problem as the linear estimator will match with MMSE estimator for the joint Gaussian case.)

1. Input arguments to the this program:
   (a) first argument and second arguments are $\mu_0$ and $\sigma_0^2$ respectively.
   (b) third argument is the csv file name. This file has three columns. The first column contains the samples, second the corresponding $\sigma_i^2$ values and third the $h_i$ values.
2. Example commands that you should test with:
   (a) python 7_ee25btech00000.py 1 10 samples.csv
   (b) python 7_ee25btech00000.py 2 100 samples.csv
3. Output of this program.
   (a) Fetch the total number of available samples $N$ from the samples.csv file. This should be the number of rows of the csv file.
   (b) Vary $n$ in the set $1, 2, \cdots, N$ and get MMSE estimates, $\hat{x}(Y_1), \hat{x}(Y_1, Y_2), \cdots, \hat{x}(Y_1, Y_2, \cdots, Y_N)$. Plot these $N$ MMSE estimates of $X$ as scatter plot. The X-axis here are values $1, 2, \cdots, N$ and the Y-axis corresponds to the estimates. Do you see the estimates getting refined with increase in $n$ ?

### 7.1 Grading

This question accounts to 30 marks

## 8 Acknowledgements

Thanks to our undergraduate TAs, Yashwanth, Agam, Faraz, Kaushik, Akshara and Shubhang who helped prepare the program assignment.

## 9 References

1. `https://algebra.math.bme.hu/2019-20-1/BMETE91AM46-A1#11`
2. Stanley Chan, Introduction to Probability for DATA SCIENCE
3. Introduction to Probability by Bertsekas and Tsitsiklis
4. A first course on probability, Sheldon M. Ross, 10th edition.